

****Food Ordering System****

1. Define an abstract class called `FoodItem` with the following attributes and methods:
- Attributes: - `name` (protected String). `price` (protected double)
 - Methods: - `display()`: an abstract method that displays the details of the food item.

```
import java.util.Scanner;
// Abstract class representing a Food Item
abstract class FoodItem {
    protected String name;
    protected double price;

    public FoodItem(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public abstract void display();

    public double getPrice() {
        return price;
    }
}
```

2. Define an interface called `Orderable` with the following methods:
- `placeOrder(int quantity)`: a method that takes an integer `quantity` as a parameter and only prints a message indicating the order details.
 - `cancelOrder()`: a method that only prints a cancellation message.

```
// Interface representing an Orderable Item
interface Orderable {
    void placeOrder(int quantity);
    void cancelOrder();
    double getPrice();
}
```

3. Implement two concrete classes, `Pizza` and `Shake`, which inherit from the `FoodItem` class and implement the `Orderable` interface.

Each class should have additional attributes and methods as follows:

- `Pizza` class:
 - Attributes: - `size` (private String): representing the size of the pizza.
 - Methods: - Implement the abstract `display()` method to display the pizza's details.
- `Shake` class:
 - Attributes: - `size` (private String): representing the size of the shake.
 - Methods: - Implement the abstract `display()` method to display the shake's details.

```
// Concrete class representing a Pizza
class Pizza extends FoodItem implements Orderable {
    private String size;
    public Pizza(String name, double price, String size) {
        super(name, price);
        this.size = size;
    }
    public void display() {
        System.out.println("Pizza: " + name + " (Size: " + size + ")");
        System.out.println("Price: Rs" + price);
    }
    public void placeOrder(int quantity) {
        System.out.println("Ordered " + quantity + " " + size + " " + name + " pizzas.");
    }
    public void cancelOrder() {
        System.out.println("Cancelled " + size + " " + name + " pizza order.");
    }
}

// Concrete class representing a Shake
class Shake extends FoodItem implements Orderable {
    private String size;
    public Shake(String name, double price, String size) {
        super(name, price);
        this.size = size;
    }
    public void display() {
        System.out.println("Shake: " + name + " (size: " + size + ")");
        System.out.println("Price: Rs" + price);
    }
    public void placeOrder(int quantity) {
        System.out.println("Ordered " + quantity + " " + size + " " + name + " shakes.");
    }
    public void cancelOrder() {
        System.out.println("Cancelled " + size + " " + name + " shake order.");
    }
}
}
```

4. Define a custom exception class called `InvalidMenuItemException` that extends the `Exception` class. The `InvalidMenuItemException` should have a constructor that accepts a String parameter to display an error message.

Handle any exceptions that may occur during the payment process using appropriate exception handling mechanisms.

```
// Custom exception class for handling invalid menu item selections
class InvalidMenuItemException extends Exception {
    public InvalidMenuItemException(String message) {
        super(message);
    }
}

// Custom exception class for handling payment failures
class PaymentFailedException extends Exception {
    public PaymentFailedException(String message) {
        super(message);
    }
}
```

5. In the `FoodOrderingSystem` class's `main` method, perform all the following steps:

- Create an array of `FoodItem` called `menu` with a capacity of 4.
- Initialize the `menu` array with four different food items, including pizzas and shakes, each with different name, price and size as follows:

Fooditem	Name	Price	Size
Pizza	Margherita	349	Regular
Pizza	Pepperoni	449	Large
Shake	Chocolate	149	Regular
Shake	Strawberry	249	Large

- Display the menu by iterating over the `menu` array and calling the `display()` method for each item.
- Prompt the user to enter the item number they wish to order.
- Validate the user input:- If the item number is invalid (less than 1 or greater than the menu length), throw an `InvalidMenuItemException` with the message "Invalid item number selected!".
- If the item number is valid, then prompt the user to enter the quantity to order.
- If no exceptions occur, retrieve the selected item from the `menu` array based on the user's input, cast it to `Orderable`, and call the `placeOrder()` method with the specified quantity.
- Prompt the user to make a payment by entering a payment method (1 for cash, 2 for credit card).
- Implement the `payment()` method with a switch case to handle the payment scenarios:

For Cash payment: Show total bill amount and ask user to enter the same or more cash_amount. If cash_amount > = bill amount, then payment is successful, display a payment confirmation message. Else throw a custom exception called `PaymentFailedException` with the message "Cash Payment failed!".

For Creditcard payment: Show total bill amount and ask user to enter the same or more amount. If credit_amount > =bill amount and Creditcard= 12345678, then only payment is successful, display a payment confirmation message. Else throw a custom exception called `PaymentFailedException` with the message "Creditcard Payment failed!".

Handle any exceptions that may occur during the payment process using appropriate exception handling mechanisms.

```
public class FoodOrderingSystem {
    static Scanner scanner = new Scanner(System.in);
    public static void main(String[] args) {

        // Menu Items
        FoodItem[] menu = new FoodItem[4];

        menu[0] = new Pizza("Margherita", 349, "Regular");
        menu[1] = new Pizza("Pepperoni", 449, "Large");
        menu[2] = new Shake("Chocolate", 149, "Regular");
        menu[3] = new Shake("Strawberry", 249, "Large");

        // Display the menu
        System.out.println("Welcome to the Food Ordering System!");
        System.out.println("Menu:");
        for (int i = 0; i < menu.length; i++) {
            System.out.print((i + 1) + ". ");
            menu[i].display();
            System.out.println();
        }

        try {

            // Get user input for item selection
            System.out.print("Enter the item number you wish to order: ");
            int itemNumber = scanner.nextInt();

            // Validate the item number
            if (itemNumber < 1 || itemNumber > menu.length) {
                throw new InvalidMenuItemException("Invalid item number selected!");
            }

            // Get user input for quantity
            System.out.print("Enter the quantity you wish to order: ");
            int quantity = scanner.nextInt();
            // Place the order
            Orderable selectedItem = (Orderable) menu[itemNumber - 1];
//casting
            selectedItem.placeOrder(quantity);
        }
    }
}
```

```

// Get user input for payment method
System.out.println("Select a payment method:");
System.out.println("1. Cash");
System.out.println("2. Credit Card");

System.out.print("Enter the payment method: ");
int paymentMethod = scanner.nextInt();

// Process the payment
payment(selectedItem.getPrice() * quantity, paymentMethod);
System.out.println("Payment successful! Order placed successfully.");

} catch (InvalidMenuItemException e) {
    System.out.println("Error: " + e.getMessage());
} catch (PaymentFailedException e) {
    System.out.println("Error: " + e.getMessage());
}
}

// Method to process payment
public static void payment(double amount, int paymentMethod) throws PaymentFailedException
{
    switch (paymentMethod) {
        case 1:
            // Cash payment
            System.out.println("Your total bill is:" + amount);
            System.out.println("Enter bill Amount to pay:");
            double cash = scanner.nextDouble();
            if (cash < amount) {
                throw new PaymentFailedException("Cash Payment failed!");
            }
            break;
        case 2:
            // Credit card payment
            System.out.println("Your total bill is:" + amount);
            System.out.println("Enter bill Amount to pay:");
            double credit = scanner.nextDouble();
            if (credit < amount) {
                throw new PaymentFailedException("Creditcard Payment failed!");
            }
            System.out.println("Enter Credit card number:(valid: 12345678)");
            int creditcard = scanner.nextInt();
            if (creditcard != 12345678) {

```

```
        throw new PaymentFailedException("Creditcard Payment failed!");
    }
    break;

default:
    throw new PaymentFailedException("Payment Method invalid!");
}
}
```